Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

ЗВІТ

з лабораторної роботи № 4
з дисципліни «Методи та засоби продуктової аналітики»

**Тема: «AI-підкріплена дата-аналітика:
побудова власного аналітичного асистента»**

Виконав:
студент групи ІМ-31
Соломка Андрій

Київ — 2026

Мета роботи

Побудувати робочий прототип AI-асистента продуктового аналітика — інструмент, який забирає на себе рутинну частину роботи з даними: профілювання нового датасету, пошук сегментів з аномальною поведінкою, відповіді на разові питання та підготовку тексту для стейкхолдера. Окремим завданням є критичне ставлення до виходу LLM: інструмент має не лише користуватися моделлю, а й контролювати її.

Теоретичні відомості

Чому LLM не можна доручати розрахунки

Мовна модель не виконує арифметику — вона продовжує послідовність токенів за найімовірнішим продовженням. Коли модель «рахує» середній чек, вона насправді генерує число, яке виглядає правдоподібно для такого контексту. Для аналітики це критично: помилка на 10% у числі виглядає так само впевнено, як і правильне значення, і не супроводжується жодним сигналом невпевненості.

Звідси випливає архітектурний принцип, на якому побудований увесь інструмент: усі числа рахує код, модель лише інтерпретує готові числа. Модель жодного разу не отримує сирий CSV — вона бачить JSON із уже порахованими фактами. Це звужує простір, у якому вона може помилитися, до формулювань.

Галюцинації та їх перевірка

Навіть отримавши готові факти, модель може назвати число, якого в них немає: перемішати два показники, округлити в інший бік або просто вставити правдоподібне значення. Найнебезпечніший випадок — не очевидна дурниця, а число поруч із правильним (3,7% проти 4,1%), яке не викликає підозри і йде далі у звіт.

Тому промпт тут — не захист, а лише зменшення ймовірності. Поверх нього потрібна програмна перевірка: з тексту відповіді витягуються всі числа і звіряються з множиною чисел, які реально є у фактах. Складність у тому, що збіг має бути не точним: модель законно пише «139,8 тис.» замість 139 793 або «4,32%» замість 0,0432, тому перевірка враховує округлення та перерахунок у тисячі й відсотки.

Структуровані виходи

Витягувати числа і смислові блоки з вільного тексту ненадійно: формат відповіді змінюється від виклику до виклику. Тому всі функції інструменту вимагають від моделі JSON за заданою схемою. На рівні API це підкріплюється режимом JSON (response_format в OpenAI), а для Anthropic використовується

префіл відповіді: якщо повідомлення асистента почати символом «{», модель фізично не може написати вступ на кшталт «Звичайно! Ось ваш JSON».

Безпека згенерованого коду

Функція «питання природною мовою → код» означає виконання коду, який написала модель. Це класична вразливість: достатньо, щоб у даних або в питанні опинилася інструкція, і модель згенерує звернення до файлової системи чи мережі. Тому згенерований код проходить чорний список небезпечних конструкцій і виконується з урізаним набором вбудованих функцій та на копії датафрейму.

Окремо варто розрізняти два джерела помилок. Промпт зменшує кількість поганих генерацій, але не гарантує їх відсутності; перевірка коду ловить те, що прослизнуло. Обидва механізми потрібні одночасно саме тому, що закривають різні ризики.

Декомпозиція зміни метрики

Щоб пояснити, чому метрика змінилась, недостатньо показати її динаміку. Потрібен внесок кожного сегмента: наскільки змінилось значення всередині групи і яку частку загальної зміни ця група дала. Внесок рахується як відношення зміни в сегменті до загальної зміни; сума внесків дорівнює 100%, а сегменти, що рухались у протилежний бік, дають від'ємний внесок. Саме ця таблиця й відповідає на питання «хто винен», а модель формулює гіпотезу причини вже поверх неї.

Хід роботи

Архітектура інструменту

Інструмент названо DataPilot і реалізовано як Python-пакет із CLI. Логіка всюди однакова: код рахує → модель пояснює → код перевіряє.

```
datapilot/
  llm.py          шар доступу до LLM: Anthropic / OpenAI / mock, парсинг JSON
  profiling.py    детерміноване профілювання, прапорці якості даних
  insights.py     зрізи по сегментах, кореляції, денні аномалії за z-score
  eda.py          функція 1 — автоматичний EDA
  hypotheses.py   функція 2 — генерація гіпотез
  nl2pandas.py    функція 3 — NL -> pandas + пісочниця виконання
  anomaly.py      функція 4 — пояснення зміни метрики
  summary.py      функція 5 — executive summary
  verify.py       власна функція — звірка чисел моделі з фактами
  cli.py          інтерфейс командного рядка
data/             демонстраційний датасет
demo/             ноутбук із прогоном і збережені виводи команд
tests/            тести на ризиковані місця
```

Лістинг 1 — структура репозиторію

Шар доступу до моделі винесено окремо, щоб інструмент не був прив'язаний до одного провайдера: усі функції викликають complete_json, яка повертає вже розпарсений словник. Провайдер визначається автоматично за наявним ключем, а без ключа вмикається режим mock — це дозволяє запускати тести й демонструвати детерміновану частину без доступу до API.

Демонстраційні дані

Інструмент тестувався на навчальній базі мобільного застосунку за 5 квітня – 11 травня 2024 року — тій самій, що у лабораторній №3. З сирих таблиць (понад 120 МБ) зібрано три аналітичні зрізи: users.csv (60 000 користувачів із реєстрацією, платформою, країною, A/B-групою, агрегованою активністю та виручкою), daily_metrics.csv (7 825 рядків «день × платформа × країна») та payments.csv (11 330 платежів). Датасет обрано свідомо: у ньому є реальна аномалія — реєстрації на iOS зупиняються після 20 квітня, — на якій видно, чи справді працює функція пояснення аномалій.

Функція 1. Автоматичний EDA

Профілювання повністю детерміноване: типи колонок, пропуски, унікальні значення, квартилі, викиди за правилом 1,5·IQR, дублікати рядків. Поверх цього працюють евристики якості даних — і це теж код, а не модель.

```
(df: pd.DataFrame, profile: list[dict]) -> list[str]:
    """Евристики якості даних. Їх перевіряє код, а не модель."""
    flags = []
    n = len(df)
    dup = int(df.duplicated().sum())
    if dup:
        flags.append(f"Повні дублікати рядків: {dup} ({round(dup / n * 100, 2)}%)")
    for p in profile:
        if p["missing_pct"] > 20:
            flags.append(f"Колонка '{p['column']}': {p['missing_pct']}% пропусків")
        if p["kind"] == "categorical" and p["unique"] == 1:
            flags.append(f"Колонка '{p['column']}' має одне унікальне значення — марна для аналі
  зу")
        if p["kind"] == "numeric" and p.get("zeros_pct", 0) > 60:
            flags.append(f"Колонка '{p['column']}': {p['zeros_pct']}% нулів — перевірте, чи це н
  е пропуски")
        if p["kind"] == "numeric" and p.get("outliers_iqr", 0) > 0:
            share = round(p["outliers_iqr"] / n * 100, 2)
            if share > 5:
                flags.append(f"Колонка '{p['column']}': {p['outliers_iqr']} викидів за IQR ({sha
  re}%)")
        if p["kind"] == "numeric" and p.get("min", 0) < 0 and any(
                w in p["column"].lower() for w in ("cnt", "count", "amount", "revenue", "duratio
  n", "age")):
            flags.append(f"Колонка '{p['column']}' має від'ємні значення, хоча за змістом не пов
  инна")
    if not flags:
        flags.append("Грубих проблем якості не виявлено")
    return flags
```

Лістинг 2 — евристики якості даних (фрагмент profiling.py)

```
Профіль датасету
Рядків: 7,825   Колонок: 11   Дублікатів: 0   Пам'ять: 1.39 МБ

Колонки
        колонка         тип  пропуски, %  унікальних
           date    datetime          0.0          37
            app categorical          0.0           5
   country_name categorical          0.0         196
       sessions     numeric          0.0         777
session_minutes     numeric          0.0        4956
   active_users     numeric          0.0         542
      new_users     numeric          0.0         323
    revenue_usd     numeric          0.0         671
         payers     numeric          0.0          67
     activities     numeric          0.0        2670
avg_session_min     numeric          0.0        2194

Прапорці якості даних (пораховано кодом)
 • Колонка 'sessions': 1282 викидів за IQR (16.38%)
 • Колонка 'session_minutes': 1263 викидів за IQR (16.14%)
 • Колонка 'active_users': 1355 викидів за IQR (17.32%)
 • Колонка 'new_users': 1238 викидів за IQR (15.82%)
 • Колонка 'revenue_usd': 81.97% нулів — перевірте, чи це не пропуски
 • Колонка 'revenue_usd': 1411 викидів за IQR (18.03%)
 • Колонка 'payers': 81.97% нулів — перевірте, чи це не пропуски
 • Колонка 'payers': 1411 викидів за IQR (18.03%)
 • Колонка 'activities': 1369 викидів за IQR (17.5%)
 • Колонка 'avg_session_min': 1530 викидів за IQR (19.55%)
```

Лістинг 3 — вивід команди eda на daily_metrics.csv

Модель отримує цей JSON і повертає інтерпретацію за фіксованою схемою: огляд, проблеми якості, ключові спостереження, наступні кроки. Прапорець --no-llm вимикає звернення до моделі й лишає тільки розрахунки.

Функція 2. Генерація гіпотез

Щоб гіпотези були предметними, моделі недостатньо дати df.describe() — він описує колонки загалом і не показує різниці між сегментами. Тому код рахує зрізи: середні значення ключових метрик по кожній категорійній колонці, розкид між найкращою і найгіршою групою з відношенням, кореляції та денну динаміку з відхиленнями понад 2,5 сигми. Метрики для зрізів обираються не підряд, а за пріоритетом — спершу гроші й утримання, інакше гіпотези виходять про вік і технічні лічильники.

У схемі відповіді підстава винесена в окреме обов'язкове поле evidence. Це головне рішення цієї функції: коли для числа є спеціальне місце, модель його заповнює, і гіпотезу стає можливо перевірити. Поле how_to_check змушує одразу думати про спростовність.

Функція 3. Питання природною мовою → pandas

Модель отримує схему датафрейму — назви колонок, типи, діапазони числових і до десяти реальних значень категорійних. Останнє виявилось принциповим: без прикладів значень модель пише df[df.app == 'iOS'], тоді як у даних ios у нижньому регістрі, і запит мовчки повертає нуль рядків — відповідь є, виглядає правдоподібно і є неправильною.

```
(code: str) -> None:
    low = code.lower()
    for bad in FORBIDDEN:
        if bad in low:
            raise UnsafeCode(f"У згенерованому коді є заборонена конструкція: {bad!r}")
    if "result" not in code:
        raise UnsafeCode("Код не присвоює змінну `result`")


def run_code(code: str, df: pd.DataFrame):
    check_code(code)
    env = {"df": df.copy(), "pd": pd, "np": np}
    exec(compile(textwrap.dedent(code), "<datapilot>", "exec"), {"__builtins__": {
        "len": len, "range": range, "min": min, "max": max, "sum": sum,
        "round": round, "sorted": sorted, "abs": abs, "list": list, "dict": dict,
        "set": set, "str": str, "int": int, "float": float, "bool": bool,
        "enumerate": enumerate, "zip": zip, "print": lambda *a, **k: None,
    }}, env)
    return env.get("result")
```

Лістинг 4 — перевірка та виконання згенерованого коду

Код завжди показується користувачеві разом із поясненням і припущеннями, які зробила модель, — аналітик має перевіряти логіку, а не довіряти чорній скриньці. Якщо потрібної колонки немає, модель повертає code: null і пояснення; це краще, ніж вигадана колонка і KeyError.

Функція 4. Пояснення зміни метрики

Команда explain порівнює два періоди і рахує внесок кожного сегмента у зміну. Нижче — прогін на демо-даних: притік нових користувачів після 20 квітня виріс у чотири рази, і саме на цьому тлі видно, що реєстрації на iOS впали практично до нуля.

```
Метрика new_users
Базовий період   2024-04-05..2024-04-19: 27,204.0
Порівнюваний     2024-04-20..2024-05-11: 112,589.0
Зміна: 85,385.0 (313.87%)

Внесок сегментів: app
  group    base  compare  change  change_pct  contribution_pct
 mobile 17096.0 111109.0 94013.0      549.91            110.10
    ios  8710.0     89.0 -8621.0      -98.98            -10.10
android   775.0     79.0  -696.0      -89.81             -0.82
desktop   623.0   1312.0   689.0      110.59              0.81
unknown     0.0      0.0     0.0         NaN              0.00

Внесок сегментів: country_name
                  group   base  compare  change  change_pct  contribution_pct
United States of America 5243.0  21991.0 16748.0      319.44             19.61
      Dominican Republic  903.0   8831.0  7928.0      877.96              9.29
            South Africa 2801.0  10681.0  7880.0      281.33              9.23
                    Peru  962.0   7689.0  6727.0      699.27              7.88
                 Ukraine 3738.0   8716.0  4978.0      133.17              5.83
                   Spain  916.0   5743.0  4827.0      526.97              5.65
```

Лістинг 5 — вивід команди explain для метрики new_users

Це і є той випадок, заради якого функція будувалась: загальне зростання маскує локальну поломку. У відсотках зміни iOS дає −98,98%, а його внесок у загальну зміну — −10,1% при +110,1% у mobile. Модель поверх цієї таблиці формулює гіпотезу, причому факти й припущення рознесені в схемі по різних полях, а кожна гіпотеза має рівень впевненості.

Функція 5. Executive summary

На вхід подається JSON з уже порахованими метриками, на вихід — короткий текст для керівника. Промпт забороняє жаргон аналітиків і вимагає структури «що сталось → чому це важливо → що робити», причому рекомендація має бути дією, а не спостереженням; поле owner у схемі додатково підштовхує до конкретики.

Власна функція. Верифікація чисел

Шоста функція — та, якої немає у переліку завдання, але без якої решта ненадійна. Після кожної відповіді з тексту моделі витягуються всі числа і звіряються з числами, що реально є у фактах. Допускаються округлення (139 793 → «139,8 тис.») і перерахунок у відсотки (0,0432 → «4,32%»); дрібні цілі, роки та номери пунктів ігноруються, щоб не засмічувати список.

```
ok   = "За період зареєструвалось 139 793 користувачі, виручка 93 749.58 $."
bad  = "За період зареєструвалось 139 793 користувачі, конверсія в платіж 4.1%."

OK                          <- За період зареєструвалось 139 793 користувачі, виручка 93 749.58 $.
НЕПІДТВЕРДЖЕНО [4.1]        <- За період зареєструвалось 139 793 користувачі, конверсія в платіж 4.1%
```

Лістинг 6 — верифікатор відрізняє правильне число від правдоподібного вигаданого

Реальне значення конверсії у платіж — 3,7%. Число 4,1% правдоподібне і стоїть поруч із правильним, тому без автоматичної перевірки воно спокійно пішло б у звіт. У CLI такі значення виводяться окремим попередженням.

Тестування та крайні випадки

Тести покривають саме ризиковані місця, а не тривіальні функції:

— верифікатор ловить вигадане число і не чіпає округлене чи переведене у відсотки;

— пісочниця відхиляє імпорти, читання файлів і звернення до dunder-атрибутів;

— код без присвоєння result відхиляється як такий, що не відповідає контракту;

— виконання згенерованого коду не змінює вихідний датафрейм;

— профілювання правильно рахує пропуски й піднімає відповідний прапорець;

— декомпозиція знаходить обвал iOS у демонстраційних даних.

```
$ python -m pytest tests -q
........                                                  [100%]
8 passed in 0.19s
```

Лістинг 7 — прогін тестів

Окремо перевірено поведінку на крайніх випадках: порожній CSV (інструмент завершується з поясненням, а не зі стектрейсом), відсутній API-ключ (режим mock із попередженням), запит до неіснуючої колонки (модель повертає code: null, користувач бачить пояснення), а також небезпечний код, який відхиляється перевіркою.

Репозиторій і документація

Код, демонстраційні дані, ноутбук із прогоном та збережені виводи команд зібрані в репозиторії. README містить опис, інструкцію запуску, перелік функцій, приклади використання з реальним виводом і пояснення, де взяти API-ключ. Окремий файл prompts.md містить усі промпти з розбором: що робить кожне правило, яку проблему воно закриває і чому формулювання саме таке.

Репозиторій:

https://github.com/AndriiSolomka/kpi-product-analytics-lab4

Висновки

У ході роботи побудовано DataPilot — CLI-асистента продуктового аналітика, який реалізує п'ять функцій із завдання (автоматичний EDA, генерацію гіпотез,

запити природною мовою, пояснення аномалій та executive summary) і одну власну — верифікацію чисел у відповідях моделі.

Головний висновок стосується розподілу ролей. Спокуса віддати LLM весь аналіз велика, але модель не рахує — вона генерує правдоподібне. Робочою виявилась схема, в якій LLM не бачить сирих даних взагалі: pandas рахує факти, модель перетворює їх на текст, код перевіряє текст назад проти фактів. При такому поділі модель може помилитися лише у формулюванні, а не в числі, і навіть ця помилка ловиться автоматично.

Другий висновок — про промпти. Найбільший ефект дали не ввічливі прохання, а зміни структури відповіді: винесення підстави гіпотези в окреме обов'язкове поле, розділення фактів і припущень на різні поля схеми, дозвіл явно відповісти «не можу». Схема формує поведінку сильніше, ніж формулювання інструкції, бо порожнє обов'язкове поле модель заповнює, а розмиту вимогу — ігнорує.

Третій висновок — практичний: на демонстраційних даних інструмент за одну команду знайшов те, що в дашборді лабораторної №3 було помітно лише при уважному розгляді окремого графіка. Притік користувачів виріс у чотири рази, і це зростання маскувало зупинку реєстрацій на iOS; декомпозиція внесків показала це одразу — −98,98% у сегменті, який дав −10,1% до загальної зміни. Саме на таких задачах AI-асистент економить аналітику ті 80% рутини, про які йдеться в завданні.